AI Career Coach

Existing Flaws, Practical Fixes & Hackathon Differentiation

Research and solution-planning brief | October 2026

Executive takeaway. Stand out by linking real internship requirements to verifiable GitHub code, tracking uncertainty, and automatically revising affected assessments after real commits. Do not pitch generic AI career advice as the innovation.

1. Ten existing gaps and proposed solutions

Potential gap	Solution design	Priority
Resume-dependent assessment	Examine source code instead of trusting skill claims.	Critical
Static skill-gap reports	Use GitHub push webhooks for automatic reassessment.	Critical
Unverifiable AI ratings	Link files, functions, code lines and commit hashes.	Critical
Generic job recommendations	Compare repository evidence against linked real internships.	Critical
Generic learning advice	Offer project-specific code changes and challenges.	High
No progress history	Store assessments and explain differences across commits.	High
Keyword-only detection	Analyze implementation, test coverage and complexity.	High
Unclear uncertainty	Distinguish demonstrated, partial and not demonstrated.	Critical
Unclear authorship	Check contribution signals; note limits of ownership claims.	Medium
Costly rescans	Identify changed files and reassess affected skills.	High

Qualification: These are common pitfalls or differentiation opportunities, not deficiencies verified in every individual competitor.

2. Five essential improvements

A. Source Code Intelligence Engine

Inspect real logic rather than repository names, README keywords or language proportions. For example, the existence of **database.py** does not prove SQL ability: require actual queries, operations, or tests. Record what the evidence proves and its limitations.

B. Automatic Skill Reassessment

Subscribe to GitHub push events, verify webhook signatures, fetch changes, map affected files to skills, and update relevant assessments. Show the commit and before/after evidence. Handle deletions as well as additions.

C. Explainable Skill Evidence

For every skill claim show repository, file path, line range, commit SHA, job requirement and reason. Avoid unsupported proficiency percentages. If no evidence exists, say **not demonstrated in the reviewed scope**, not **student cannot do it**.

D. Job-to-Project Gap Resolver

Connect a missing job requirement to an actionable change in an existing project. Example: add persistent SQL storage and test coverage to a Python expense tracker; then reassess when code changes are pushed.

E. Career Readiness History

Show the evolution of evidence over time, including increases, decreases, unclear cases and unchanged skills. Preserve older assessments for a transparent audit trail.

3. Example: a real commit updates SQL evidence

Stage	Python	SQL	REST APIs
Initial assessment	Demonstrated	Not demonstrated	Partial
Commit arrives	Unchanged	New SQL queries + test	Unchanged
Updated assessment	Demonstrated	Query writing demonstrated	Partial

Demo loop: real internship posting requiring Python, SQL and REST APIs → inspect connected repository → record gap → push **database/queries.sql** and a test → receive a webhook → reassess SQL → display cited updated evidence.

Important nuance: adding a few SQL queries demonstrates specific query-writing capabilities; it does not establish mastery of SQL or production database administration.

4. Competitor landscape and room to differentiate

Project	Documented focus	Opportunity
GitRoll	GitHub-based developer assessment	Student-facing job requirements with transparent commit-level reassessment
SkillProof	GitHub evidence, skill verdicts, job matching, improvement tasks	Reliable live reassessment and a visible audit trail
SkillBridge	Resume-to-job analysis and career roadmaps	Source-code-grounded skill evidence
Skill Gap Analysis	Jobs-to-candidate skill comparisons	Continuously updated repository evidence

SkillProof is particularly relevant: source-file references and actionable skill-gap tasks are already part of its public design. They are baseline requirements, not wholly novel features.

5. Recommended implementation stack

Component	Technology	Purpose
Frontend	Next.js + Tailwind CSS	Student dashboards
Backend	Python + FastAPI	API, parsing and orchestration
GitHub integration	GitHub App/API + webhooks	Repository and change detection
Evidence analysis	Tree-sitter + code-capable LLM	Structured parsing and evidence explanations
Storage	Supabase / PostgreSQL	Skills, links and history
Processing	Job queue; Celery/Redis if needed	Reliable asynchronous webhook processing
Job requirements	Employer postings / permitted job APIs	Citable current requirements

For a hackathon, begin with Next.js, FastAPI, a real GitHub webhook, an LLM with rule-based checks, and Supabase. Avoid extra infrastructure unless required.

6. Judging and development priorities

Feature	Suggested priority
Working commit-to-assessment loop	5 / 5
Verifiable file and commit evidence	5 / 5
Real job requirements and source links	5 / 5
Progress timeline	4 / 5
Project-specific challenges	3 / 5
AI resume generation	2 / 5

These are subjective development priorities rather than official judge scores. Prioritize functional completeness, execution quality, thought, effort and creativity.

7. Demo plan and key pitch

> *Traditional career platforms tell students which skills to learn. Our AI Career Coach shows what their GitHub code actually demonstrates, compares that evidence with real internship requirements, and automatically revises the assessment when their repositories change.*

- Show a real internship page and save its source URL.
- Connect a real GitHub repository and display file/line evidence and an unproven SQL requirement.
- Make a genuine commit adding query code and a test; show the push event.
- Display automatic reassessment, new citations, and assessment-history change.
- Explain limitations: code does not always prove authorship or mastery; missing evidence is not missing ability.

8. Reference links

GitRoll: https://gitroll.io/
SkillProof: https://github.com/aryancodex-19/T36-SkillProof
SkillBridge: https://github.com/prithvifr/SkillBridge
Skill Gap Analysis: https://github.com/skill-gap-dev/skill_gap_analysis
GitHub webhook documentation: GitHub push events and payloads

Bottom line: Build the continuous GitHub evidence loop first. A convincing before-and-after live demonstration is more valuable than many optional features without end-to-end integration.